

TECH

REACT CORE

RENDERING HOOKS EFFECTS **REACT 18+ / CONCURRENCY**

REACT 18+ / CONCURRENCY

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК react.dev

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	useTransition Q1 · Q2 · Q3	URGENT vs TRANSITION	Как помечать обновления как «не срочные» и почему это меняет UX.
ДЕНЬ 2 СРЕДНИЙ	useDeferredValue useTransition Q4 · Q5 · Q6	vs DEBOUNCE vs	Второй инструмент приоритетов и в чём он отличается от соседей.
ДЕНЬ 3 СРЕДНИЙ	SUSPENSE Q7 · Q8 · Q9	ЗАЧЕМ РИСКИ	Что Suspense даёт и какие границы стоит знать.
ДЕНЬ 4 ТЯЖЁЛЫЙ	useSyncExternalStore S Q10 · Q11 · Q12	EXTERNAL STORE	Как корректно подключить внешний store к concurrent-режиму.
ДЕНЬ 5 ТЯЖЁЛЫЙ	CONCURRENT-COMPATIBLE STORE MODE Q13 · Q14	STRICT	Что должен уметь store, чтобы не ломать React 18, и как Strict ловит баги.
ДЕНЬ 6 ТЯЖЁЛЫЙ	TEARING Q15 · Q16 · Q17	SUBSCRIPTION-BASED STATE	Главный enemy concurrent-режима и как его не получить.
ДЕНЬ 7 СРЕДНИЙ	EFFECTS + CONCURRENCY OVERKILL Q18 · Q19 · Q20	ЧТО НЕ РЕШАЕТ	Тонкости и трезвый взгляд: где concurrency помогает, а где — нет.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

useTransition URGENT vs TRANSITION

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как помечать обновления как «не срочные» и почему это меняет UX.

ВОПРОСЫ ДНЯ

Q01 Что такое useTransition?

Q02 Когда использовать startTransition?

Q03 Чем urgent update отличается от transition update?

Что такое useTransition?

ОПИСАНИЕ

useTransition – хук React 18, который позволяет пометить обновление как «не срочное». Возвращает [isPending, startTransition]. Всё, что обернули в startTransition, получает низкий приоритет: React может прервать такой рендер, если пришло срочное обновление (например, ввод). Пока transition в процессе, isPending = true – удобно для локального индикатора.

КАК РАБОТАЕТ

- 01 `const [isPending, startTransition] = useTransition();`
- 02 `startTransition(callback)`: все `setState` внутри `callback` получают transition-приоритет.
- 03 `isPending = true` пока React работает над transition.
- 04 Срочные обновления (ввод) не задерживаются – рендерятся сразу.
- 05 При новом срочном апдейте текущий transition прерывается и перезапускается с актуальными данными.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Поиск с фильтрацией большого списка: ввод – urgent, фильтр – transition.
- 02 Переключение между тяжёлыми табами/вкладками.
- 03 Сортировка большой таблицы.
- 04 Любой UI, где есть быстрая «голова» и медленный «хвост».

ПОДВОДНЫЕ КАМНИ

- 01 useTransition не уменьшает CPU – просто переставляет приоритеты. Тяжёлый рендер останется тяжёлым.
- 02 Не подходит для async-работы – startTransition внутри `await` не действует на код после `await`.
- 03 Если внутри transition нет `setState` – ничего не поменяется.

КОД

// Поиск с фильтром

```
function Search() {
  const [q, setQ] = useState('');
  const [list, setList] = useState<User[]>([]);
  const [isPending, startTransition] = useTransition();

  const onChange = (e: ChangeEvent<HTMLInputElement>) => {
    setQ(e.target.value); // urgent
    startTransition(() => {
```

```
        setList(filter(e.target.value));          // transition
    });
};

return (
  <>
    <input value={q} onChange={onChange} />
    {isPending && <span>filtering...</span>}
    <List items={list} />
  </>
);
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что произойдёт, если новый urgent апдейт прервёт transition?
- 02 Чем useTransition отличается от useDeferredValue?
- 03 Можно ли использовать startTransition без useTransition?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useTransition>

Когда использовать startTransition?

ОПИСАНИЕ

`startTransition` – глобальный аналог `useTransition` (без `isPending`). Используется там, где не нужен локальный флаг ожидания, а просто хочется пометить апдейт как не-срочный: при роутинге, отправке аналитики, обновлении кэша. Также удобен в обработчиках событий вне компонента (например, в `onClick` глобальной кнопки из роутера).

КАК РАБОТАЕТ

- 01 `import { startTransition } from 'react'.`
- 02 `startTransition(callback)` – пометить `setState`'ы внутри `callback` как `low-priority`.
- 03 Без `isPending` – не блокирует рендер локального индикатора.
- 04 Часто заворачивают навигацию: `startTransition(() => router.push('/x'))`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Навигация в Next/Remix-роутере.
- 02 Обновление кэша или `background-store`.
- 03 Внутри `event-handler` без UI-фидбека.
- 04 Внутри библиотечного кода, где нет компонента.

ПОДВОДНЫЕ КАМНИ

- 01 Если нужен индикатор `pending` – использовать `useTransition`.
- 02 `startTransition` не работает с `async` внутри `await`.
- 03 Не оборачивать `setState` с `urgent`-семантикой (ввод, `клик-responsiveness`).

КОД

// Навигация

```
import { startTransition } from 'react';
function MenuItem({ to }: { to: string }) {
  return (
    <a onClick={e => {
      e.preventDefault();
      startTransition(() => router.push(to));
    }}>...</a>
  );
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `startTransition` отличается от `useTransition`?
- 02 Когда не нужен `isPending`?

03 Что произойдёт, если в `startTransition` сделать `setState`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/startTransition>

Чем urgent update отличается от transition update?

ОПИСАНИЕ

Urgent – высокоприоритетное обновление, которое React обрабатывает сразу: ввод в input, клик, hover. Transition – низкоприоритетное, которое React может отложить, прервать или пересобрать, чтобы не мешать срочным. Принцип: пользователь должен видеть мгновенную реакцию на ввод; обновление списка по новому запросу – не должно лагать ввод.

КАК РАБОТАЕТ

- 01 Urgent: setState вне startTransition. По умолчанию все обновления urgent.
- 02 Transition: setState внутри startTransition.
- 03 При коллизии urgent побеждает: React сначала отрисует urgent, потом подтянет transition.
- 04 Прерывание: при новом urgent текущий transition перезапускается с самого начала.
- 05 Внутри transition можно делать несколько setState – все батчатся.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Urgent: контролируемые input, клики, переключатели.
- 02 Transition: фильтрация списка, тяжёлый перерасчёт UI, изменение route.
- 03 Это деление – основа respons-of UI в React 18.

ПОДВОДНЫЕ КАМНИ

- 01 Urgent setState внутри transition – TS не предупредит, просто потеряется приоритет (smerges).
- 02 Если всё пометить transition – UI станет «вялым» (любая реакция отложена).
- 03 isPending – единственный способ показать пользователю, что transition идёт.

КОД

// Сравнение приоритетов

```
// urgent – реагирует мгновенно
setQuery(e.target.value);

// transition – может ждать, не блокирует следующий ввод
startTransition(() => setResults(filter(e.target.value)));
```

МОГУТ ДОСПРОСИТЬ

- 01 Что произойдёт при двух одновременных transition?
- 02 Можно ли отменить transition?
- 03 Где найти список urgent-событий?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useTransition>

useDeferredValue vs DEBOUNCE vs useTransition

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Второй инструмент приоритетов и в чём он отличается от соседей.

ВОПРОСЫ ДНЯ

Q04 Что такое useDeferredValue?

Q05 Чем useDeferredValue отличается от debounce?

Q06 Чем useDeferredValue отличается от useTransition?

ВОПРОС Q04

Что такое `useDeferredValue`?

ОПИСАНИЕ

`useDeferredValue` – хук, возвращающий «отложенную» версию переданного значения. Когда `value` меняется, хук сначала возвращает старое, а потом, в `low-priority transition`, переключается на новое. Полезен, когда нет контроля над тем, кто делает `setState` (например, проп приходит из родителя), а локально хочется отложить тяжёлый рендер.

КАК РАБОТАЕТ

- 01 `const deferred = useDeferredValue(value).`
- 02 `deferred` сначала равен старому `value`, потом «догоняет».
- 03 Новое значение применяется внутри `transition` – может прерываться.
- 04 Полезно сравнивать `deferred !== value` для показа индикатора "stale".
- 05 В отличие от `useTransition` не требует доступа к месту `setState`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Тяжёлый список, фильтруемый по приходу пропа.
- 02 Чарт/карта, реагирующая на изменения внешнего параметра.
- 03 Когда `startTransition` недоступен (значение приходит извне).

ПОДВОДНЫЕ КАМНИ

- 01 Не уменьшает работу – просто переставляет приоритеты.
- 02 Если `update` быстрее, чем рендер успевает завершиться – `deferred` может застрять старым.
- 03 Не подходит для `async`-данных (это не `fetch`-кеш).

КОД

// Stale-индикатор

```
function Search({ query }: { query: string }) {
  const deferred = useDeferredValue(query);
  const isStale = deferred !== query;
  return (
    <div style={{ opacity: isStale ? 0.5 : 1 }}>
      <List items={filter(deferred)} />
    </div>
  );
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `deferred` отличается от `debounce`?
- 02 Можно ли использовать с объектом-значением?

03 Что произойдёт при частом изменении?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useDeferredValue>

ВОПРОС Q05

Чем `useDeferredValue` отличается от `debounce`?

ОПИСАНИЕ

`Debounce` — таймер: ждём N мс паузы, потом вызываем. `useDeferredValue` — приоритеты: значение обновляется сразу, но рендер с новым значением идёт с `low-priority`. Главное отличие: `debounce` задерживает САМ ВЫЗОВ функции (например, `fetch`), а `deferred` задерживает РЕНДЕР. `Debounce` работает с временем; `deferred` — с нагрузкой.

КАК РАБОТАЕТ

- 01 `Debounce`: `setTimeout/clearTimeout`, ожидание паузы.
- 02 `useDeferredValue`: React сам решает, когда переключиться на новое значение, на основе доступного времени.
- 03 `Debounce` подходит для I/O — реже дёргать сервер.
- 04 `Deferred` подходит для CPU — реже перерисовывать тяжёлый UI.
- 05 Их можно комбинировать: `debounce` для запроса, `deferred` для UI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Поиск к серверу: `debounce`.
- 02 Тяжёлый локальный рендер: `useDeferredValue`.
- 03 Combo: `debounce` → запрос, `deferred` → UI с результатом.

ПОДВОДНЫЕ КАМНИ

- 01 Использовать `deferred` для редких сетевых запросов — запросы будут лететь на каждый ввод.
- 02 Использовать `debounce` для UI — пользователь не увидит промежуточных состояний.
- 03 Думать, что `deferred` «магически быстрее» — он только переставляет приоритеты.

КОД

// Combo

```
function Search() {
  const [q, setQ] = useState('');
  const debounced = useDebounceValue(q, 300);
  const { data } = useQuery(['search', debounced], () =>
    api.search(debounced)
  );
  const deferred = useDeferredValue(data ?? []);
  return <List items={deferred} />;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда `deferred` НЕ помогает?

- 02 Можно ли заменить debounce на deferred в input?
- 03 Чем deferred отличается от throttle?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useDeferredValue>

ВОПРОС Q06

Чем `useDeferredValue` отличается от `useTransition`?

ОПИСАНИЕ

`useTransition` – обёртка вокруг `setState`, которую вы контролируете в момент изменения. `useDeferredValue` – обёртка вокруг ЗНАЧЕНИЯ, которая откладывает реакцию на его изменение. Если у вас есть доступ к `setState` – берите `useTransition` (явно и точно). Если значение приходит снаружи (prop, контекст, store) – `useDeferredValue`.

КАК РАБОТАЕТ

- 01 `useTransition`: вы оборачиваете `setState`. Получаете `isPending`.
- 02 `useDeferredValue`: вы оборачиваете значение. Получаете отложенную копию.
- 03 Эффект схожий: рендер с новым состоянием идёт с `low-priority`.
- 04 `useTransition` даёт `isPending`; `deferred` – нет, но можно сравнить `deferred !== value`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `useTransition`: `setState` под вашим контролем.
- 02 `useDeferredValue`: значение приходит из `props/context/store`.
- 03 Часто оба применяют в одном компоненте по разным поводам.

ПОДВОДНЫЕ КАМНИ

- 01 Если можно использовать `useTransition` – он точнее, потому что охватывает только нужные `setState`.
- 02 `useDeferredValue` с примитивом – дешево; с объектом – идентичность через `===`, и React сравнивает по ссылке.
- 03 Иногда сочетают: `useTransition` для action + `deferred` для отображения внешних данных.

КОД

// Когда что

```
// есть доступ к setState – useTransition
const [isPending, startTransition] = useTransition();
startTransition(() => setItems(big));

// значение приходит как prop – useDeferredValue
function List({ filter }: { filter: string }) {
  const deferred = useDeferredValue(filter);
  return <Heavy filter={deferred} />;
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Можно ли совместить оба хука в одном компоненте?
- 02 Что выбрать, если есть и `setState`, и проп?
- 03 Что покажет `isPending` во время `deferred`-обновления?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useDeferredValue#deferring-re-rendering-for-a-part-of-the-ui>

ДЕНЬ 3 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

SUSPENSE ЗАЧЕМ РИСКИ

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Что Suspense даёт и какие границы стоит знать.

ВОПРОСЫ ДНЯ

- Q07 Когда использовать Suspense?
- Q08 Какие проблемы решает Suspense?
- Q09 Какие риски у Suspense?

Когда использовать Suspense?

ОПИСАНИЕ

Suspense – границы загрузки в React-дереве. `<Suspense fallback={<Spinner />}>...</Suspense>` показывает fallback, пока кто-то внутри границы «приостанавливает» себя (через `bundle-split`, через асинх-данные с поддержкой Suspense, через `React.lazy`). Полезно, когда хочется управлять loading-состояниями декларативно, без `if (loading)` во многих местах.

КАК РАБОТАЕТ

- 01 Компонент внутри Suspense может «бросить» промис – React покажет fallback и подождёт.
- 02 `React.lazy` + Suspense – основной способ code-splitting для роутов.
- 03 Асинх-данные через библиотеки (`react-query` 5+, `swr` с `suspense:true`) интегрируются с Suspense.
- 04 Концерция `SuspenseList` (experimental) управляет порядком показа нескольких fallback'ов.
- 05 Streaming SSR – отдельная сильная сторона Suspense.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code-splitting роутов и тяжёлых компонентов.
- 02 Streaming SSR в Next App Router (`loading.tsx`).
- 03 React-Query с suspense-режимом для declarative loading.
- 04 Image-loading с `suspense-aware`-обёртками.

ПОДВОДНЫЕ КАМНИ

- 01 Не каждая библиотека данных поддерживает Suspense – проверять.
- 02 Слишком крупная граница Suspense – пользователь видит длинный спиннер.
- 03 Слишком мелкая – много отдельных flickers.
- 04 Без `error-boundary` рядом – необработанные ошибки в `promise` делают компонент сломанным.

КОД

// Code-splitting + lazy

```
const Heavy = React.lazy(() => import('./Heavy'));
function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Heavy />
    </Suspense>
  );
}
```

```
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие библиотеки данных поддерживают Suspense?
- 02 Чем Suspense лучше `if (loading)`?
- 03 Зачем `error-boundary` вместе с Suspense?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/Suspense>

Какие проблемы решает Suspense?

ОПИСАНИЕ

Suspense унифицирует loading-states: вместо разных if (isLoading) в каждом компоненте – одна декларативная граница с fallback. Также решает waterfalls: при правильной интеграции данные начинают грузиться параллельно, а не последовательно. И позволяет streaming-рендер с прогрессивной выдачей UI пользователю.

КАК РАБОТАЕТ

- 01 Декларативный loading – один fallback на много загрузок внутри.
- 02 Streaming SSR: сервер отдаёт разметку по частям, fallback показывается, пока часть не готова.
- 03 Concurrent-rendering позволяет показывать частичный UI и догружать остальное.
- 04 Selective hydration – гидрация ветки, как только JS+данные приехали.
- 05 Suspense + Transitions – управление приоритетом loading'a.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большие SPA с много данных на одной странице.
- 02 Next App Router (loading.tsx, error.tsx).
- 03 Микрофронтенды с разными источниками данных.

ПОДВОДНЫЕ КАМНИ

- 01 Без поддержки в data-layer Suspense бесполезен (нужна интеграция в react-query/swr/relay).
- 02 Каскад Suspense – fallback на fallback'e (UI «мерцает»).
- 03 Промис, бросаемый в render, должен быть стабилен между рендерами – иначе бесконечный цикл.

КОД

```
// react-query suspense
```

```
const { data } = useSuspenseQuery({
  queryKey: ['users'],
  queryFn: api.users,
});
// data – гарантировано определён, никаких if (loading)
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое waterfalls и как Suspense с ними борется?
- 02 Чем streaming SSR отличается от обычного?
- 03 Какие данные не подходят для Suspense?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/Suspense>

Какие риски у Suspense?

ОПИСАНИЕ

Suspense прячет сложность асинхронности – это и плюс, и минус. Минусы: каскад загрузок (несколько Suspense вложены – спиннер за спиннером), unstable promise (новый промис при каждом рендере → бесконечный цикл), race condition (старый промис вернётся после нового и затрёт данные), необходимость error-boundary рядом, сложности с тестированием.

КАК РАБОТАЕТ

- 01 Suspense ловит брошенный промис – но если при каждом рендере промис новый, цикл не разорвётся.
- 02 Race condition: при смене props новый запрос; если старый вернётся позже – переписать (если нет version-защиты).
- 03 Errors внутри Suspense должен ловить error-boundary, иначе всё дерево падает.
- 04 Тестирование: jest нужно использовать act + waitForSuspense.
- 05 В DevTools профилировать сложнее – рендеры идут пакетами.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Понимание помогает дизайннить data-layer.
- 02 Code review компонентов с Suspense – проверять на stable promise.
- 03 Архитектура roots: где разместить error-boundary, где Suspense.

ПОДВОДНЫЕ КАМНИ

- 01 Stable promise – самая частая ошибка. Использовать react-query/swr, не свой fetch.
- 02 Слишком крупная граница – UX страдает.
- 03 Слишком мелкая – flicker.
- 04 Не забывать про error-boundary.

КОД

// Antipattern – нестабильный промис

```
function Bad() {
  const promise = fetch('/x');           // каждый рендер новый!
  use(promise);                          // бесконечный suspense
  return <div />;
}
```

// Стабильный – через библиотеку

```
const { data } = useSuspenseQuery({ queryKey: ['x'], queryFn: api.x });
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое use-хук в RSC?
- 02 Как тестировать Suspense?
- 03 Как отлавливать ошибки внутри Suspense?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/Suspense#caveats>

useSyncExternalStore

EXTERNAL STORES

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как корректно подключить внешний store к concurrent-режиму.

ВОПРОСЫ ДНЯ

Q10 Что такое useSyncExternalStore?

Q11 Когда нужен useSyncExternalStore?

Q12 Почему external store нельзя просто читать из module variable?

ВОПРОС Q10

Что такое useSyncExternalStore?

ОПИСАНИЕ

`useSyncExternalStore` – хук React 18, предназначенный для правильной интеграции внешних источников данных (Redux, Zustand, browser APIs, custom event-bus). Принимает `subscribe`, `getSnapshot` и опциональный `getServerSnapshot`. Гарантирует, что в concurrent-режиме React не покажет несогласованные данные (tearing) и корректно подпишется/отпишется при изменениях.

КАК РАБОТАЕТ

- 01 `subscribe(notify)`: React вызывает с callback'ом, store должен вызвать `notify` при каждом изменении и вернуть `unsubscribe`.
- 02 `getSnapshot()`: синхронно вернуть текущее состояние. Должно возвращать стабильную ссылку, если ничего не поменялось (`Object.is`).
- 03 `getServerSnapshot()`: для SSR. Если не задано – будет ошибка при server render.
- 04 React сравнивает snapshot через `Object.is`. Меняется ссылка → ререндер consumer'а.
- 05 Под капотом отслеживает версию между render и commit, чтобы избежать tearing.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Кастомные store (Redux до новых биндингов, Zustand, Mobx-обёртки).
- 02 Browser APIs: `window-size`, `online-status`, `localStorage`.
- 03 Любой shared mutable state, который читается несколькими компонентами.

ПОДВОДНЫЕ КАМНИ

- 01 `getSnapshot` должен возвращать стабильную ссылку – иначе infinite render.
- 02 `subscribe` должен возвращать `unsubscribe`.
- 03 Без `getServerSnapshot` SSR упадёт.
- 04 Селекторы внутри `getSnapshot` – должны мемоизировать результат.

КОД

// Подписка на window.online

```
function useOnline() {
  return useSyncExternalStore(
    (cb) => {
      window.addEventListener('online', cb);
      window.addEventListener('offline', cb);
      return () => {
        window.removeEventListener('online', cb);
        window.removeEventListener('offline', cb);
      };
    },
```

```
    () => navigator.onLine,  
    () => true,  
  );  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое tearing и при чём тут useSyncExternalStore?
- 02 Зачем нужен getServerSnapshot?
- 03 Можно ли использовать с селекторами?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

ВОПРОС Q11

Когда нужен `useSyncExternalStore`?

ОПИСАНИЕ

Когда есть внешний (вне React-дерева) источник данных, который меняется во времени и нужно подписать на него компоненты так, чтобы в concurrent-режиме UI оставался согласованным. Это includes: глобальные store, browser APIs, любые pub-sub. В обычном режиме можно обходиться `useState` + `useEffect`, но получится tearing.

КАК РАБОТАЕТ

- 01 Любой источник, который не управляется React-state.
- 02 Используется библиотеками: `react-redux v8+`, `zustand`, `mobx-react-lite`, `valtio`.
- 03 Прямой клиентский код тоже может – для browser APIs и custom store.
- 04 Альтернатива (`useState`+`useEffect`) ломается при concurrent rendering: state читается до commit, потом меняется в external store, UI tear.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Глобальный state (`Redux/Zustand/MobX`).
- 02 Browser APIs: `matchMedia`, `online`, `scroll`, `geolocation`.
- 03 `WebSocket` / `event-bus` / `observer`.
- 04 Custom event emitters (ваша pubsub-обёртка).

ПОДВОДНЫЕ КАМНИ

- 01 Если данные внутри React (`useState`) – `useSyncExternalStore` не нужен.
- 02 Высокочастотные обновления (`scroll`, `mouse-move`) – `throttling/debounce` обязательно, иначе ререндер на каждое событие.
- 03 `selector`'ы должны мемоизировать результат – иначе infinite re-render.

КОД

```
// useMatchMedia
```

```
function useMediaQuery(q: string) {
  return useSyncExternalStore(
    (cb) => {
      const m = window.matchMedia(q);
      m.addEventListener('change', cb);
      return () => m.removeEventListener('change', cb);
    },
    () => window.matchMedia(q).matches,
    () => false,
  );
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем react-redux v8 отличается от v7?
- 02 Когда useState достаточно?
- 03 Как добавить selector с мемо?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

ВОПРОС Q12

Почему `external store` нельзя просто читать из `module variable`?

ОПИСАНИЕ

Если в `render`-функции читать обычную `module`-переменную (`let store = {...}`), React не знает, что данные поменялись – нет уведомления, ререндер не случится. Хуже: в `concurrent`-режиме разные части UI могут прочитать переменную в разные моменты времени и увидеть разные значения – это и есть `tearing`. `useSyncExternalStore` решает обе проблемы: уведомляет об изменениях и обеспечивает консистентный `snapshot`.

КАК РАБОТАЕТ

- 01 Чтение `module`-переменной в `render` = вне React-реактивности.
- 02 React не подписан – ререндер не случится при изменении.
- 03 В `concurrent`: `render` может прерываться. Перебегающие чтения дадут разные значения.
- 04 `useSyncExternalStore` делает чтение через `snapshot` и проверяет согласованность.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Когда хочется сделать «простой `store`» через `let var` – это анти-паттерн в React 18.
- 02 Лучше: `useSyncExternalStore` или библиотека, которая под капотом его использует.

ПОДВОДНЫЕ КАМНИ

- 01 Без подписки UI не реагирует на изменение – баги «данные обновлены, но не отображаются».
- 02 `Tearing`: компонент A показывает старое, компонент B – новое.
- 03 Сложно дебажить – баги случаются под нагрузкой.

КОД

// Антипаттерн

```
let store = { count: 0 };
function Bad() {
  return <div>{store.count}</div>; // не реагирует
}
```

// Корректно

```
const store = createStore({ count: 0 });
function Good() {
  const count = useSyncExternalStore(
    store.subscribe,
    () => store.getState().count,
  );
  return <div>{count}</div>;
}
```

```
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое tearing на конкретном примере?
- 02 Почему concurrent делает проблему хуже?
- 03 Как исторически решали эту задачу до useSyncExternalStore?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 2 ВОПРОСА

CONCURRENT-COMPATIBLE STORE STRICT MODE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Что должен уметь store, чтобы не ломать React 18, и как Strict ловит баги.

ВОПРОСЫ ДНЯ

Q13 Как сделать store compatible with concurrent rendering?

Q14 Как React Strict Mode помогает находить проблемы?

Как сделать store compatible with concurrent rendering?

ОПИСАНИЕ

Концепция: store должен предоставлять стабильный snapshot, механизм подписки и нотификации, и эти примитивы должны интегрироваться через useSyncExternalStore. Дополнительно: selector'ы должны быть мемоизированы (равные входы → равные ссылки), мутации должны быть синхронные и атомарные, для SSR – getServerSnapshot.

КАК РАБОТАЕТ

- 01 subscribe(cb): store вызывает cb на каждое изменение, возвращает unsubscribe.
- 02 getSnapshot(): синхронно вернуть текущее состояние; стабильная ссылка для одинаковых данных.
- 03 getServerSnapshot(): для SSR – что отдавать на сервере.
- 04 Селекторы: useSyncExternalStoreWithSelector (npm пакет) – добавляет мемоизацию + сравнение результата.
- 05 Mutate: store должен обновлять snapshot и вызывать subscribers синхронно.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Кастомный store для micro-frontend / shared state.
- 02 Обёртки вокруг WebSocket / event-bus.
- 03 Свой реактивный кэш данных.

ПОДВОДНЫЕ КАМНИ

- 01 Если getSnapshot возвращает новый объект каждый раз – infinite re-render.
- 02 Если subscribe не обеспечивает unsubscribe – leak.
- 03 Async mutate (через таймер) – пропустит notification, если не вызвать subscribers вручную.

КОД

// Минимальный store

```
function createStore<T>(initial: T) {
  let state = initial;
  const listeners = new Set<() => void>();
  return {
    getState: () => state,
    setState: (next: T) => { state = next; listeners.forEach(l => l()); },
    subscribe: (cb: () => void) => { listeners.add(cb); return () => listeners.delete(cb); },
  };
}
```


МОГУТ ДОСПРОСИТЬ

- 01 Что такое `useSyncExternalStoreWithSelector`?
- 02 Как добавить `middleware`?
- 03 Как тестировать?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

Как React Strict Mode помогает находить проблемы?

ОПИСАНИЕ

Strict Mode – обёртка `<React.StrictMode>...</React.StrictMode>`, которая в development делает дополнительные проверки: вызывает render компонента дважды, перезапускает effects (mount → unmount → mount), запрещает legacy API. Цель – поймать побочные эффекты в render и effects без cleanup, пока проект ещё в разработке.

КАК РАБОТАЕТ

- 01 Render компонента вызывается дважды – побочные эффекты в render будут видны (двойной log, двойной push в массив).
- 02 useEffect: эффект выполняется → cleanup → снова эффект. Это симулирует размонтирование/перемонтирование.
- 03 Только в development. В production не активно.
- 04 Strict Mode не ломает корректный код – он его проверяет.
- 05 Legacy refs (string refs), findDOMNode и т.п. вызывают warnings.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любое приложение в dev должно быть обёрнуто.
- 02 Тесты с Strict Mode (не везде включают) – ловят больше багов.
- 03 Onboarding нового кода под Strict.

ПОДВОДНЫЕ КАМНИ

- 01 Двойной запрос в dev – нормально, если код корректен (с AbortController).
- 02 Двойной render – путает счётчики и логи.
- 03 Side effects в render проявляются громко – иногда разработчики думают, что это баг React.
- 04 В production двойного запуска нет – баги, скрытые без Strict, всплывают в проде.

КОД

// Включаем Strict

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что Strict Mode не ловит?

02 Почему эффект выполняется дважды в dev?

03 Включать ли Strict в тестах?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/StrictMode>

TEARING**SUBSCRIPTION-BASED STATE****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Главный енему concurrent-режима и как его не получить.

ВОПРОСЫ ДНЯ

Q15 Что такое tearing?

Q16 Как избежать tearing?

Q17 Как проектировать subscription-based state?

ВОПРОС Q15

Что такое tearing?

ОПИСАНИЕ

Tearing – рассинхронизация UI: одна часть экрана показывает старое состояние, другая – новое, потому что они прочитали данные в разные моменты во время concurrent render. Это видимая «трещина» по UI. Возникает, когда внешний store меняется между двумя чтениями его в render-фазе. До React 18 не было проблемой, потому что render был неразрывен.

КАК РАБОТАЕТ

- 01 Concurrent render может прерваться, потом продолжиться.
- 02 Между прерыванием и продолжением внешний store мог поменяться.
- 03 Если разные компоненты читают store в разное время – видят разные значения.
- 04 useSyncExternalStore проверяет, что snapshot не поменялся между render и commit – если поменялся, перезапускает render.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Понимание помогает диагностировать «странные» баги в React 18 после миграции.
- 02 При проектировании своего store.
- 03 При выборе библиотеки state-management.

ПОДВОДНЫЕ КАМНИ

- 01 Без useSyncExternalStore tearing проявляется случайно – трудно воспроизвести.
- 02 В тестах часто не виден – нужны concurrent-rendering сценарии.
- 03 Старые версии redux/mobx могли страдать; современные – нет.

КОД

// Симптом

```
// в одном компоненте
<UserName /> // 'Alice'
// в другом, ниже по дереву
<UserAge /> // 30 (старое; настоящее – 31)
```

МОГУТ ДОСПРОСИТЬ

- 01 Почему tearing появился именно в React 18?
- 02 Как useSyncExternalStore это лечит?
- 03 Можно ли воспроизвести tearing в тесте?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

Как избежать tearing?

ОПИСАНИЕ

Использовать `useSyncExternalStore` (или библиотеку, которая его использует под капотом – `react-redux v8+`, `Zustand v4+`). Не читать внешние данные через `let`-переменную в `render`. Делать обновления `store` синхронными (или `batched`). Для `async`-данных – `react-query/swr` (они уже `concurrent-safe`).

КАК РАБОТАЕТ

- 01 `useSyncExternalStore`: обязательный механизм для внешнего `store`.
- 02 `subscribe` + `getSnapshot` – два примитива, которые делают снэпшот атомарным.
- 03 `Store` должен обновлять `snapshot` синхронно при `mutate`, потом вызвать `subscribers`.
- 04 Проверять, что библиотека `stateя` – `concurrent-safe`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой нетривиальный `state` (`Redux/Zustand/MobX/custom`).
- 02 `Browser APIs`.
- 03 Пользовательские `event-bus`.

ПОДВОДНЫЕ КАМНИ

- 01 Использование старой версии `state-library` (`redux <v8`) – потенциальный `tearing`.
- 02 Чтение из `module-variable` в `render`.
- 03 `Async mutate` без `notify subscribers` – данные обновлены, `UI` нет.

КОД

// Правильно

```
import { useSyncExternalStore } from 'react';
function useStoreValue<S, T>(store, selector) {
  return useSyncExternalStore(
    store.subscribe,
    () => selector(store.getState()),
  );
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие библиотеки уже `concurrent-safe`?
- 02 Что делать с неподдерживаемыми `API`?
- 03 Как написать `selector` с мемо?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

Как проектировать subscription-based state?

ОПИСАНИЕ

Базовая модель: store предоставляет subscribe / getSnapshot / (getServerSnapshot) / mutate-методы. Mutate должен синхронно обновить состояние и вызвать subscribers. Snapshot должен быть стабильной ссылкой при равных данных. Селекторы – отдельный слой с мемо. Подписка – только на изменения, которые затрагивают конкретный selector.

КАК РАБОТАЕТ

- 01 Минимум API: getState, setState, subscribe.
- 02 subscribe возвращает unsubscribe – без этого утечки.
- 03 Atomic update: state = next; subscribers.forEach(fn => fn());
- 04 Selector pattern: useSelector(store, s => s.user.name) → ререндер только при смене name.
- 05 Поддержка middleware (logging, devtools, persistence).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Кастомные глобальные state-менеджеры.
- 02 Микро-frontend shared state.
- 03 Domain-stores в больших приложениях.
- 04 Адаптеры внешних библиотек (e.g. shopping-cart-sdk).

ПОДВОДНЫЕ КАМНИ

- 01 Без atomic mutate – рассинхрон.
- 02 Без stable snapshot – infinite re-render.
- 03 Без selector мемо – лишние ререндеры.
- 04 Без unsubscribe – утечки и устаревшие listeners.

КОД

// Простой store

```
function createStore<T>(initial: T) {
  let state = initial;
  const subs = new Set<() => void>();
  return {
    getState: () => state,
    setState: (next: T) => {
      if (Object.is(state, next)) return;
      state = next;
      subs.forEach(fn => fn());
    },
    subscribe: (fn: () => void) => {
```



```
    subs.add(fn);  
    return () => subs.delete(fn);  
  },  
};  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем zustand отличается от такого минимального store?
- 02 Как добавить middleware?
- 03 Как реализовать time-travel debugging?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

EFFECTS + CONCURRENCY**ЧТО НЕ РЕШАЕТ****OVERKILL****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Тонкости и трезвый взгляд: где concurrency помогает, а где — нет.

ВОПРОСЫ ДНЯ

- Q18 Как concurrent rendering влияет на side effects?
- Q19 Какие performance-проблемы не решаются concurrency APIs?
- Q20 Когда concurrency API будет overkill?

ВОПРОС Q18

Как concurrent rendering влияет на side effects?

ОПИСАНИЕ

Concurrent rendering требует, чтобы render был чистым – без побочных эффектов. Все side effects живут в useEffect / useLayoutEffect / event handlers. В Strict Mode dev эффекты выполняются дважды (mount → cleanup → mount) – это симулирует возможность того, что в concurrent React фактически может пере-монтировать компонент. Это требование делает effects idempotent (см. соседнюю секцию).

КАК РАБОТАЕТ

- 01 Render обязан быть pure – никаких side effects.
- 02 Effects вызываются после commit и могут быть перезапущены.
- 03 Strict Mode dev симулирует remount – ловит effects без cleanup.
- 04 Production не делает двойной запуск – но при Suspense / возврате на страницу remount возможен по-настоящему.
- 05 Subscriptions, timers, fetches – обязательно cleanup.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой effect с подпиской/таймером/fetch.
- 02 Cleanup-функция = «отменить то, что эффект сделал».
- 03 AbortController для fetch внутри эффекта.

ПОДВОДНЫЕ КАМНИ

- 01 Эффект без cleanup – leak / двойная подписка в Strict.
- 02 Side effects в render – двойные/тройные эффекты.
- 03 Counter в effect без cleanup – растёт быстрее ожидаемого.
- 04 fetch без abort – race condition.

КОД

// Правильный effect с cleanup

```
useEffect(() => {
  const ctrl = new AbortController();
  fetch('/x', { signal: ctrl.signal }).then(...);
  return () => ctrl.abort();
}, []);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что значит «idempotent effect»?
- 02 Почему production не делает двойной mount?
- 03 Что такое cleanup корректное?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useState#my-effect-runs-twice-when-the-component-mounts>

ВОПРОС Q19

Какие performance-проблемы не решаются concurrency APIs?

ОПИСАНИЕ

Concurrency перераспределяет приоритеты, но не уменьшает общую работу CPU. Если рендер действительно тяжёлый, concurrency только размажет его во времени, а не ускорит. Такие проблемы лечатся другими способами: виртуализация списков (react-window/react-virtual), мемоизация селекторов, code-splitting, оптимизация алгоритмов, web workers для CPU-задач, server components.

КАК РАБОТАЕТ

- 01 useTransition / useDeferredValue – приоритеты, не ускорение.
- 02 Тяжёлый чистый CPU-код – Web Worker / OffscreenCanvas.
- 03 Огромный список – виртуализация (react-window).
- 04 Лишние ререндеры – React.memo / useMemo / useCallback (избирательно).
- 05 Большой bundle – code-splitting через React.lazy.
- 06 Дорогая инициализация – useMemo с пустыми deps.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Профилировать перед оптимизацией.
- 02 Выбирать инструмент по типу проблемы.
- 03 Code review – отличать «реальное узкое место» от «premature optimization».

ПОДВОДНЫЕ КАМНИ

- 01 Думать, что useTransition «делает рендер быстрее».
- 02 Мемоизация – не дефолт, нужна там, где доказана выгода.
- 03 Виртуализация – сложна; упустить корректность реактивности легко.
- 04 Web Worker – отдельный поток, общение через postMessage → overhead.

КОД

// Виртуализация

```
import { FixedSizeList as List } from 'react-window';  
<List height={500} itemCount={1e5} itemSize={30} width="100%">  
  {( { index, style } ) => <div style={style}>{items[index]}</div>}  
</List>
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда нужен Web Worker?
- 02 Чем виртуализация лучше pagination?
- 03 Как мерить CPU-нагрузку рендера?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useTransition#preventing-unwanted-loading-indicators>

ВОПРОС Q20

Когда concurrency API будет overkill?

ОПИСАНИЕ

Маленькие приложения, лёгкие компоненты, простые формы – не нуждаются в `useTransition` / `useDeferredValue` / `Suspense`. Это инструменты для крупных UI с нетривиальной нагрузкой. Внедрять их «на всякий случай» – лишний код, сложнее поддерживать, равно нулевой выигрыш в перформансе. Принцип – измерить, потом оптимизировать.

КАК РАБОТАЕТ

- 01 Concurrency реально нужен на тяжёлом рендере (1000+ элементов, графики, дашборды).
- 02 `Suspense` – для нетривиальных `loading-states` или `code-splitting`.
- 03 Tier-1 проекты могут полностью обходиться `useState` + `useEffect` + `react-query`.
- 04 Метрика: профайлер показывает `long-tasks > 50ms` – есть что прерывать.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Не использовать без причины.
- 02 Использовать, когда измерили проблему и `concurrent` – понятный путь её решения.
- 03 Не путать «новый API в `react.dev`» с «обязательно применять».

ПОДВОДНЫЕ КАМНИ

- 01 `useTransition` в каждом обработчике – `code bloat`.
- 02 `Suspense` с кастомным `data-loader` без интеграции – `infinite spinner`.
- 03 `useDeferredValue` с лёгким рендером – нет эффекта, только сложность.
- 04 Архитектурные решения «под все возможные сценарии» = `over-engineering`.

КОД

// Не используйте, если не нужно

```
// плохо – useTransition без причины
const [, startTransition] = useTransition();
<button onClick={() => startTransition(() => setCount(c + 1))}>
  +1
</button>
// просто setCount(c + 1) – достаточно
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда concurrency реально оправдан?
- 02 Как определить, что нужен `useTransition`?
- 03 Чем grand идея «прерываемого рендера» иногда продаётся не по делу?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useTransition>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Объяснить `useTransition` / `startTransition` / `urgent` vs `transition` (Q1, Q2, Q3)
- ☐ Описать `useDeferredValue` и сравнить с `debounce` и `useTransition` (Q4, Q5, Q6)
- ☐ Сказать, когда `Suspense` уместен и какие у него риски (Q7, Q8, Q9)
- ☐ Описать `useSyncExternalStore` – зачем, как и где (Q10, Q11, Q12)
- ☐ Спроектировать `concurrent-compatible store` (Q13, Q17)
- ☐ Объяснить, что такое `tearing` и как от него защищаться (Q15, Q16)
- ☐ Сказать, как `Strict Mode` помогает находить проблемы (Q14)
- ☐ Описать связь `concurrent rendering` и `side effects` (Q18)
- ☐ Назвать 3 проблемы, которые `concurrency` не решает (Q19, Q20)

EVENT LOOP / ASYNC МЕТОДИЧКА v1
МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru